

Advanced NLP – Exercise 1

Dan Abergel

May 2026

GitHub repository: https://github.com/DanAbergel/ANLP_EX1

1 Open Questions

Question 1 – Three QA datasets that annotate intrinsic concepts

By “intrinsic” I understand datasets where the QA format is just a tool to annotate something about the language, and the answer always sits inside the input sentence – so you don’t need any world knowledge to answer.

1. **QA-SRL** (He et al., 2015) – the labels are semantic roles, written as wh-questions about each verb. For “*The pilot landed the plane*” the verb *landed* gives *Who landed something?* → *the pilot* and *What did someone land?* → *the plane*. It’s intrinsic because the targets are predicate–argument relations, which are a property of the sentence itself.
2. **QAMR** (Michael et al., 2018) – same idea but more general: the meaning of a sentence is captured by a whole set of free-form QA pairs (arguments, modifiers, etc.). Same reason as QA-SRL for being intrinsic – the questions describe what the sentence says, not facts about the world.
3. **QADiscourse** (Pyatkin et al., 2020) – here the label is the discourse relation between two clauses (cause, contrast, condition...), encoded as a question whose answer reveals it. Intrinsic because what is being annotated is how the clauses connect inside the text.

Question 2 – Inference-time scaling methods

The idea of inference-time scaling is to spend more compute at test time (rather than during training) to get better outputs. From the lecture I noted three methods.

(a) The three methods

1. Self-Consistency (Wang et al., 2022).

- *Description.* Sample N different chains of thought from the same prompt (with non-zero temperature), then take the majority vote on the final answer.
- *Advantages.* Very easy to plug on top of any CoT model – no extra training, no extra model. Improves accuracy on reasoning tasks pretty reliably.
- *Bottlenecks.* You pay roughly N times the decoding cost, and you need to hold N KV caches at the same time.
- *Parallelizable?* Yes, easily. The N samples don’t depend on each other so you just put them in one big batch.

2. Verifier-based Best-of- N (Cobbe et al., 2021; Lightman et al., 2023).

- *Description.* Same idea (sample N candidates), but instead of voting you score every candidate with a separately trained verifier and keep the one with the best score.
- *Advantages.* Can still pick the correct answer when only a minority of the N samples is right – which majority voting cannot do.
- *Bottlenecks.* You pay the $N \times$ generation cost like Self-Consistency, plus N verifier forward passes. Also the verifier itself has to fit in memory next to the generator.
- *Parallelizable?* Yes, both the generation and the verifier scoring are independent across the N candidates.

3. Long chain-of-thought (OpenAI o1, 2024; DeepSeek-R1, 2025).

- *Description.* The model is trained (using RL with a verifiable reward) to produce a single but very long chain of thought, where it plans, backtracks and evaluates itself. We scale the test-time compute by letting it “think” for many more tokens.
- *Advantages.* Currently the best results on hard reasoning. And there is no external verifier or vote needed at inference time.
- *Bottlenecks.* Generation is sequential, so the KV cache and the latency grow with the number of reasoning tokens.
- *Parallelizable?* Not inside a single trajectory – you cannot parallelize an autoregressive decoding. You can still batch across different prompts.

(b) Single large-memory GPU, complex scientific task

I would go with the long-CoT model. The reason is that what limits long CoT is the size of the KV cache, and that is exactly what a large-memory GPU provides. The other two methods scale by sampling more candidates, and that benefits more from several GPUs (or just from a higher batch throughput) than from extra memory on a single one. We also saw in the lecture that on hard reasoning tasks (AIME, science olympiads), models that learn to plan and backtrack do better than sample-and-vote methods at the same compute budget.

Self-Consistency would also be a reasonable choice on one GPU, since the N samples can be batched together. But for a hard scientific task, I think one well-trained long chain that plans and self-corrects buys more than voting between N shorter ones.

2 Programming Exercise – MRPC Paraphrase Detection

I fine-tuned `bert-base-uncased` on MRPC. The whole code is in `ex1.py` in the repo: it loads the model with `AutoModelForSequenceClassification`, uses dynamic padding via `DataCollatorWithPadding`, and logs the loss to W&B at every step. The W&B project (3 runs) is here: <https://wandb.ai/danabergel1995-hebrew-university-of-jerusalem/anlp-ex1-mrpc>.

Hyper-parameter search

I tried three configurations, all on the full train and validation splits:

#	Epochs	Learning rate	Batch size	Validation acc
1	3	$2 \cdot 10^{-5}$	16	0.8701
2	3	$5 \cdot 10^{-5}$	32	0.8358
3	3	$1 \cdot 10^{-4}$	32	0.8480

Config 1 is the best, config 2 the worst, and all three are above the 75% threshold.

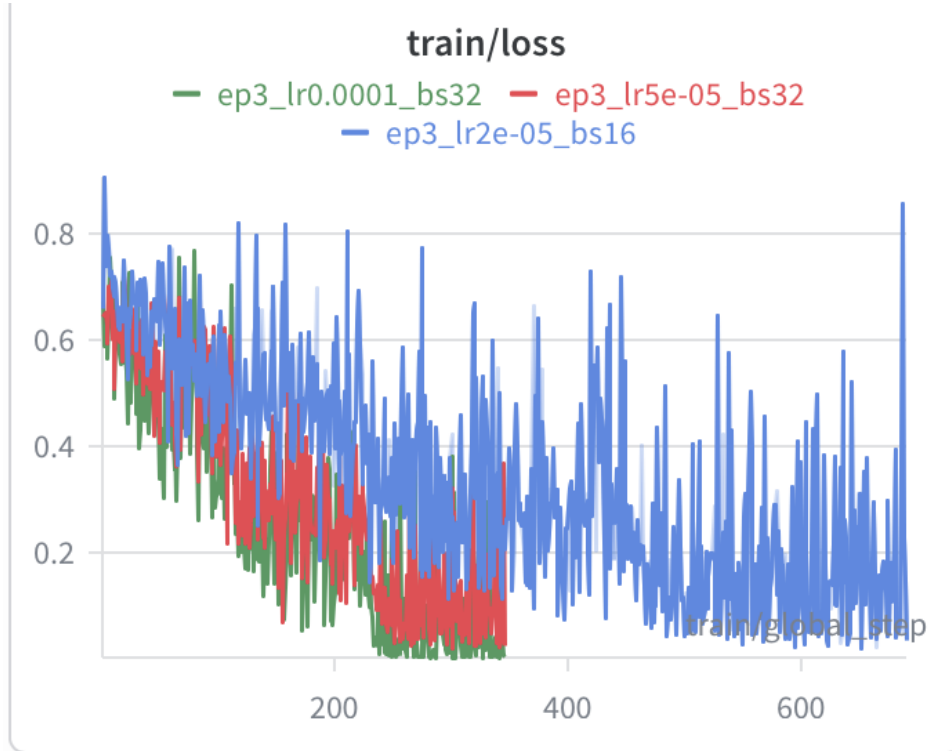


Figure 1: Train loss per step for the three configurations.

Best validation $\Rightarrow$ best test?

The test accuracies of the three checkpoints are 0.8458, 0.8354 and 0.8365 (in the same order as the table). So the best-on-validation config is also the best on test. One thing I noticed though: the gap between the configs shrinks a lot on the test set (about 1 point, vs. 3 points on validation), so the difference between the two non-winning configs on validation is partly noise.

Qualitative analysis: best vs. worst configuration

I compared config 1 (best, 87.0%) and config 2 (worst, 83.6%) on the validation set. There are 29 examples where the best is correct and the worst is wrong, and looking at them, two types of cases come back often:

- Pairs that share many words but are *not* paraphrases. The worst model seems to rely on word overlap and predicts “paraphrase”.
- Real paraphrases written with different words. The worst model sees little overlap and goes the other way.

Two examples below.

Example 1 (gold = 0, worst predicts 1). *s1*: “After losing as much as 84.56 earlier, the Dow Jones industrial average closed up 22.81, or 0.2 percent, at 9,340.45.”
s2: “In midday trading, the Dow Jones industrial average lost 68.84, or 0.7 percent, to 9,248.80.”
Both sentences are about the Dow on the same day and use similar words, but all the numbers are different. The worst model gets fooled by the overlap.

Example 2 (gold = 1, worst predicts 0). *s1*: “Broomhead was assigned to 2nd Squadron, 3rd Armor Cavalry Regiment, based at Fort Carson.”
s2: “Broomhead, 34, was assigned to the 2nd Squadron, 3rd Armored Cavalry Regiment.”
Same fact (same assignment), but *s1* mentions Fort Carson and *s2* adds his age. Those small added/missing pieces are enough to make the worst model say “not paraphrase”.

Example 3 (gold = 0, worst predicts 1). *s1*: “‘It is bad for Symbian,’ said Per Lindberg, analyst at Dresdner Kleinwort Wasserstein.”
s2: “‘Motorola has displayed clear disloyalty’ to Symbian, said Per Lindberg, an analyst at Dresdner Kleinwort Wasserstein in London.”
Same speaker, same affiliation, very similar framing – but the actual quote in each sentence is different. The worst model seems to anchor on the speaker info and miss that the two reported statements are not the same.